

LAB 4 – DATA PROCESSING USING SQL

CONTENT

- Using SQL to prepare intermediate data storage solution for future analytical needs
 - Please design and implement the entire process
 - Please prepare a short report highlighting the assumed approach and covering all topics indicated within the following tasks
- Using SQL Server 20XX Import and Export Data tool

TUTORIAL 1 – SQL QUERY EXECUTION PLAN ANALYSIS IN MICROSOFT SQL SERVER

Analyzing SQL query execution plans is a crucial skill for anyone working with MS SQL Server, especially for database administrators, developers, and performance tuners. An execution plan is a roadmap created by the SQL Server Query Optimizer that outlines the steps it intends to take to retrieve or modify data based on your SQL query. Understanding and analyzing these plans allows you to identify performance bottlenecks and optimize your queries for faster execution.

SQL Server Query Optimizer analyzes your SQL query and considers various factors like available indexes, data distribution, and statistics to determine the most efficient way to execute it. The result of this optimization process is an execution plan, which is a tree-like structure of operators. Each operator represents a specific action performed by the database engine, such as scanning a table, joining two tables, sorting data, or inserting records. Finally, the plan shows the order in which these operators will be executed.

SQL Server Management Studio provides you with two types of execution plans: estimated and actual. The primary difference lies in when they are generated and the level of detail they provide. You can think of Estimated Plan as the Query Optimizer's "plan on paper" before it starts working. It's a guess based on the information it has (like statistics, indices). Whereas Actual Plan as a "report card" after the query has finished running. It shows exactly what happened and how long each step took. In general, you use the estimated execution plan when you want a quick overview of how SQL Server thinks it will execute your query (useful for initial analysis and understanding the basic steps involved), and actual execution plan when you need to diagnose performance problems (as runtime statistics provide concrete evidence of where the bottlenecks are). In particular, you can directly see which operators took the most time or processed the most data.

We Analyze Execution Plans to identify slow-performing queries and pinpoint the exact steps causing the delay. Focusing on identifying Bottlenecks, i.e., execution plans reveal inefficient operations like full table scans when an index could be used, inefficient join algorithms, unnecessary sorting, and missing indexes. We can also use execution plans to understand query behavior and troubleshooting issues (e.g. when a query suddenly starts running slower). Finally, we can use execution plans to compare different query approaches, i.e., You can directly compare the execution plans of different versions of the same query to see which one performs better. The latter is a really neat feature, as you can run multiple queries in a single run and compare their execution times and costs.

There are several ways to obtain an execution plan in SQL Server Management Studio (SSMS):

- **Estimated Execution Plan:** You can get this by clicking the "Display Estimated Execution Plan" button in the SSMS toolbar or by using the keyboard shortcut Ctrl+L.
- **Actual Execution Plan:** You can get this by clicking the "Include Actual Execution Plan" button in the SSMS toolbar or using the keyboard shortcut Ctrl+M before executing the query. Another way is to use SET STATISTICS PROFILE ON command before executing the query. The plan will be displayed in the "Messages" tab.
- **Live Query Statistics:** This feature, available in newer versions of SQL Server, shows the execution plan as the query is running, providing real-time insights into the progress and performance of each operator. You can enable this by clicking the "Include Live Query Statistics" button in the SSMS toolbar.

Execution plans are made up of various operators, each performing a specific task. Here are some common and important operators to understand:

- **Table Scan/Clustered Index Scan:** Reads all rows in a table or clustered index. Often indicates a missing or unusable index.
- **Index Seek/Clustered Index Seek:** Efficiently retrieves specific rows using an index. This is generally preferred over a scan.

- **Key Lookup:** Occurs when a non-clustered index is used to find rows, but additional columns not included in the index are needed. This requires an extra lookup to the base table.
- **RID Lookup:** Similar to Key Lookup but used for heap tables (tables without a clustered index).
- **Sort:** Orders the data based on a specified column. Can be expensive for large datasets.
- **Hash Match/Merge Join/Nested Loops:** Different algorithms used to join two or more tables. The efficiency of each depends on the size and indexing of the tables.
- **Compute Scalar:** Performs a calculation on a column.
- **Aggregate:** Performs aggregate functions like SUM, AVG, COUNT, etc.
- **Filter:** Filters rows based on a specified condition.
- **Insert/Update/Delete:** Operators for data modification operations.

By understanding how to obtain and interpret SQL query execution plans in MS SQL Server, you can effectively diagnose and resolve performance issues, leading to faster and more efficient database applications. Remember to practice analyzing different types of queries and execution plans to build your expertise in this critical area. Below are some best practices for Execution Plan Analysis:

- **Start with the Actual Execution Plan:** as estimated plans are useful for a quick look; whilst actual plans provide runtime statistics that are crucial for accurate analysis.
- **Focus on High-Cost Operators:** Identify the operators with the highest estimated subtree cost, as these are often the primary performance bottlenecks. Note that you can examine operator properties by right-click on an operator and selecting "Properties" to view detailed information, such as the number of rows processed, the time taken, and any warnings or recommendations.
- **Look for Warnings and Errors:** Pay attention to any warning icons (yellow exclamation marks) or error icons (red crosses) in the execution plan, as they indicate potential issues.
- **Understand Join Types:** Learn about the different join algorithms (Nested Loops, Merge Join, Hash Match) and when each is most efficient. In short, Nested Loops (iterates through each row of the outer table and, for each row, searches for matching rows in the inner table) are efficient for joining small datasets or when there's a suitable index on the inner table (can be very inefficient for large, unindexed tables) – look for this join type when joining large tables without appropriate indexes; Merge Join (assumes the input tables are sorted and iteratively goes over the values) requires the input datasets to be sorted (a Sort operator might be added, which can be expensive) and is efficient when the data is already sorted or when joining large, indexed datasets; Hash Match (builds a hash table in memory for one of the tables, usually the smaller one, and then probes this hash table for matching rows from the other table) is generally used for joining large, unsorted datasets when no suitable indexes are available for a Merge Join (it can be memory-intensive). Remember that Query Optimizer chooses the join type based on its estimations. If you suspect an inefficient join type is being used, consider adding appropriate indexes or rewriting the query.
- **Analyze Data Types and Conversions:** Implicit data type conversions can sometimes lead to inefficient execution plans. Ensure data types are consistent in your queries and table definitions.
- **Consider Indexing and keep up-to-date statistics:** Analyze if missing or inefficient indexes are contributing to performance problems (as such pay attention to any index recommendations, and costly scans). Remember that outdated statistics can lead the Query Optimizer to generate suboptimal execution plans (regularly update statistics on your tables and indexes) – look for significant differences in estimated and real number of rows.
- **Test Your Changes:** After making any changes based on your execution plan analysis (e.g., adding indexes or rewriting queries), always test the performance to ensure the changes have the desired effect.

In general, the topic of query optimization is a complex one and requires more in-depth knowledge about details of operator implementation and how the storage engine is working. Yet we can pinpoint a few common performance bottlenecks that are identified in Execution Plans:

- **Table Scans/Clustered Index Scans** – in a situation when a seek is expected, indicates a missing or unusable indexes. In particular, if you apply a predication over your query and expect the system to quickly find the required values, you are expecting a seek. Now if the resultant query is doing a scan to find the predicated rows, there is a potential for a bottleneck.
- **Key Lookups/RID Lookups:** Suggests that the non-clustered index doesn't cover all the necessary columns, and you might benefit from including those columns in the index (covering index).
- **Excessive Sorting:** Look for unnecessary Sort operators, especially on large datasets. Consider if indexing can eliminate the need for sorting. Sorting can be a memory-intensive and time-consuming

operation, especially for large datasets. All the data needs to be gathered before the sorting can begin. If there isn't enough memory, SQL Server might spill data to tempdb, which drastically slows down the process.

- *Inefficient Joins (e.g., Nested Loops on large, unindexed tables):* Analyze if the join type is appropriate and if indexes can improve the join performance.
- *Missing Indexes:* SQL Server often provides index recommendations in the execution plan (look for green text hints).
- *High Estimated Subtree Cost:* Each operator in the execution plan has an estimated cost, and the cost of a subtree represents the cumulative cost of all operators within that part of the plan. Operators with a significantly high cost are potential bottlenecks, as it is indicated that those operations are expected to consume a large portion of the query's resources. Remember, that a high estimated cost doesn't always mean there's a problem, as some operations are inherently more expensive than others.
- *Spills to TempDB:* Operators like Sort or Hash Match might spill data to the tempdb (on disk) database if there isn't enough memory. This can severely impact performance, as disk I/O is significantly slower than in-memory operations. Look for warnings in the operator properties.

In the upcoming task, we will just touch on the problem of query performance analysis. Please note that this introduction was intended to pinpoint this important topic and showcase some major approaches and situations.

TUTORIAL 2 –IMPORT/EXPORT DATA

Using dedicated tools to ingest and move data around different tables and databases. Please follow this basic introduction to SQL Server Import/Export tool:

1. Using your local database server, create a new database.
 - a. <https://docs.microsoft.com/en-us/sql/relational-databases/databases/create-a-database>
2. Run SQL Server 20XX Import and Export Data (additional tool that is installed with SQL Server)
 - a. Follow the description available at
 - i. <https://docs.microsoft.com/en-us/sql/integration-services/import-export-data/steps-in-the-sql-server-import-and-export-wizard>
 - b. Additional resources
 - i. <https://docs.microsoft.com/en-us/sql/integration-services/import-export-data/import-and-export-data-with-the-sql-server-import-and-export-wizard>
 - ii. <https://www.sqlshack.com/importexport-data-sql-server-using-sql-server-import-export-wizard/>
3. You can test your knowledge (NO RESULT UPLOAD) on using SQL Server 20XX Import and Export Data tool by loading 3 tables from AdventureWorks database, namely Production.Product, Production.ProductSubCategory and Production.ProductCategory, into a different database on your local machine (please use [ProductImport], [ProductSubcategoryImport] and [ProductCategoryImport] as table names) and [DWSRC] as the schema.
 - a. Within the process try to remove 'rowguids' and 'modifieddate' attributes from each table. Remember to use proper data provider that can connect to the source database, and use Column Mapping to deselect unneeded attributes
 - b. Please verify that the table is successfully created, i.e., check whether the identity constraints are defined, check whether referential integrity is maintained.

Another approach to move data around different sources is to use **BCP (Bulk Copy Program) tool**. It is a command-line utility that ships with SQL Server. It is designed for one thing: moving massive amounts of data in or out of tables with extreme efficiency. It bypasses much of the overhead associated with standard SQL INSERT statements by using the Bulk Load API. In short, it exports data from a source table into a flat file (like .dat, .csv, or .txt) and then imports that file into the destination table. Importantly, it performs "minimal logging," meaning it doesn't record every single row insertion in the transaction log, which prevents the log from exploding in size and drastically increases speed.

BCP is an ideal tool to migrate millions of rows between different environments, automate data dumps via batch scripts or scheduled tasks, or move data across not connected networks.

Moving data with BCP is a **two-step "Out-and-In" process**: you export data from the source to a physical file, and then import that file into the destination.

Step 1: The "Out" (Export)

First, you need to export data from the source server, by creating a data file (usually a .dat or .csv).

Syntax Structure: `bcp [Table] out [File_Path] -S [Source_Server] -T -n`

- **out:** Specifies the export direction.
- **-S:** The Source Server name.
- **-T:** Uses "Trusted Connection" (Windows Auth). Use **-U** and **-P** for SQL logins.
- **-n:** Native format. This is the "secret sauce" for speed; it uses SQL's internal data types instead of converting everything to text.

Example:

```
bcp AdventureWorks.Sales.Orders out C:\Data\OrdersExport.dat -S SourceServer01 -T -n
```

Step 2: The "In" (Import)

Second, once you have the file, you can import the data, by running the utility tool again but on the destination server.

Syntax Structure: `bcp [Table] in [File_Path] -S [Dest_Server] -T -n`

- **in:** Specifies the import direction.
- **-b 5000:** (Batch size) Tells BCP to commit every 5,000 rows. If the transfer fails, you don't lose everything.
- **-h "TABLOCK":** This places a bulk-update lock on the table, significantly increasing speed by reducing transaction log overhead.

Example:

```
bcp AdventureWorks.Sales.Orders in C:\Data\OrdersExport.dat -S DestServer02 -T -n -b 5000 -h "TABLOCK"
```

Note that if your source and destination tables aren't identical (e.g., the destination has an extra column), BCP will fail. To fix this, you need to generate a **Format File**, which acts as a "map" that tells BCP: "Take column 1 from the file and put it into column 3 of the table."

To generate a map for a particular table you can use bcp with the following syntax:

```
bcp TableName format nul -f MyTable.fmt -n -S Server -T
```

Further to use the map you need to add `-f MyTable.fmt` to your in or out command.

Let us consider a simple scenario of mismatched tables, where source table has 3 columns (ID, Name, Email), and destination table has 4 columns (ID, Name, Email, and SignupDate). Assume, we have already exported data into a data file 'users.dat', which contains data for those 3 source columns.

- Note: SignupDate has a default value, or we want to skip it during the import.

First, we generate a template from the **destination** table. BCP looks at the table and creates a "map" of what it expects.

```
bcp TargetDB.dbo.Users format nul -c -f C:\Data\Users.fmt -S MyServer -T
```

- **nul:** Tells BCP we aren't exporting data, just creating the format.
- **-c:** Uses character format (common for readable files).
- **-f:** Specifies the output path for the format file.

Next, we can edit the format file by opening the 'Users.fmt' in Notepad. It will look something like this:

```
14.0
4
1 SQLCHAR 0 10 "\t" 1 ID ""
```

```
2 SQLCHAR 0 50 "\t" 2 Name ""
3 SQLCHAR 0 50 "\t" 3 Email ""
4 SQLCHAR 0 24 "\r\n" 4 SignupDate ""
```

To tell BCP "The data file only has 3 columns, ignore the 4th column in the table," you change the number of columns at the top and remove the line for the 4th column. Note, that in the example below the 4 is changed to a 3, and the delimiter of the 3rd row is updated to \r\n (newline) because it is now the end of the data file record.

```
14.0
3
1 SQLCHAR 0 10 "\t" 1 ID ""
2 SQLCHAR 0 50 "\t" 2 Name ""
3 SQLCHAR 0 50 "\r\n" 3 Email ""
```

Now, you can run the import by using the new format file in the command line using the -f flag to point to your custom map:

```
bcp TargetDB.dbo.Users in C:\Data\users.dat -f C:\Data\Users.fmt -S MyServer -T
```

Note that if you skip a column in the destination table, SQL Server will insert a NULL unless that column has a Default Constraint defined in the schema.

Alternatively, if the data file has 4 columns but the destination table only has 3 (the reverse of above), you don't delete the line. Instead, you set the Server Column Order to 0:

```
14.0
4
1 SQLCHAR 0 10 "\t" 1 ID ""
2 SQLCHAR 0 50 "\t" 0 Extra "" <-- The '0' tells BCP to ignore this data
3 SQLCHAR 0 50 "\t" 2 Name ""
4 SQLCHAR 0 50 "\r\n" 3 Email ""
```

There is also an XML format file option (-x). They are more verbose but much easier to read and less prone to "column counting" errors.

Another approach is to use **Linked Servers option**. A Linked Server configuration allows you to execute T-SQL commands against an external data source as if it were a local table. It essentially creates a permanent "shortcut" to another server. It is simple, you just need to define a connection (using OLE DB or ODBC) within your SQL Server instance. Once configured, you can use it to query the remote data, like [ServerName].[DatabaseName].[Schema].[TableName]. Importantly, it allows for Distributed Queries. You can join a local table with a remote table in a single SELECT statement, or use INSERT...SELECT to push data across the network in real-time. It is a perfect solution for occasional data synchronization between two live servers (for more rigorous synch you should use data replication options), doing cross-server reporting where data needs to be aggregated from multiple sources, and small to medium data transfers where setting up a file-based process (like BCP) is an overkill.

Setting up a Linked Server is essentially like creating a "saved connection" inside your SQL Server. Once established, you can query a remote database as if it were sitting on your own local drive. You can do this either through the **SQL Server Management Studio (SSMS) GUI** or via **T-SQL scripts**. Remember that SQL Server typically communicates over **Port 1433** – ensure the destination server's firewall allows traffic from your source server's IP. Try not to use Windows Authentication (@useself = 'True') through a linked server, as it often fails (unless you have **Kerberos** configured). Finally acknowledge the fact that Linked servers are convenient, but they aren't always the way to go – as sometimes SQL Server will pull the entire remote table across the network just to filter it locally. As such it is not a preferred way to operate on a large dataset (in such a case consider using OPENQUERY to force the filtering to happen on the remote side, like SELECT * FROM OPENQUERY([RemoteServerName], 'SELECT * FROM SalesDB.dbo.Orders).

Method 1: Using the SSMS Graphic Interface

1. Open **SSMS** and connect to your local (source) server.
2. Expand the **Server Objects** folder.
3. Right-click **Linked Servers** and select **New Linked Server...**
4. **General Page:**
 - **Linked server:** Enter the network name or IP address of the target server.
 - **Server type:** Select "SQL Server" (this simplifies the connection).
5. **Security Page:**
 - This defines how the local server logs into the remote server. The most common reliable method is to select "**Be made using this security context**" and enter a specific SQL Login (like a service account) that exists on the remote server.
6. **Server Options Page:**
 - Ensure **RPC** and **RPC Out** are set to **True** if you plan on running stored procedures across the link.
7. Click **OK**.

Method 2: Using T-SQL

If you prefer scripting, you use two system stored procedures: `sp_addlinkedserver` to create the link and `sp_addlinkedsrvlogin` to handle the credentials.

-- 1. Create the Linked Server connection

```
EXEC sp_addlinkedserver
    @server = N'RemoteServerName',
    @srvproduct = N'',
    @provider = N'SQLNCLI',
    @datasrc = N'192.168.1.100'; -- The IP or Network Name
```

-- 2. Define the security credentials

```
EXEC sp_addlinkedsrvlogin
    @rmtsrvname = N'RemoteServerName',
    @useself = N'False',
    @locallogin = NULL,
    @rmtuser = N'YourRemoteUser',
    @rmtpassword = N'YourStrongPassword';
```

How to Query the Linked Server

Once the link is created, you must use *Four-Part Naming* to access the data. The syntax is:

[Server].[Database].[Schema].[Table]

Example:

```
SELECT * FROM [RemoteServerName].[SalesDB].[dbo].[Orders] WHERE OrderDate > '2023-01-01';
```

TUTORIAL 3 – SCHEMAS, INGESTING AND MOVING DATA AROUND IN SQL

Using SQL to ingest and move data around different tables and databases:

- Refresh your knowledge using the provided links:
 - Schema creation - <https://msdn.microsoft.com/en-us/library/ms189462.aspx>
 - In T-SQL: <https://docs.microsoft.com/en-us/sql/t-sql/statements/create-schema-transact-sql>
 - In SQL Server Management Studio: <https://docs.microsoft.com/en-us/sql/relational-databases/security/authentication-access/create-a-database-schema>
 - **SELECT INTO** clause - <http://msdn.microsoft.com/en-us/library/ms188029.aspx>
 - **INSERT INTO...SELECT** clause - <https://learn.microsoft.com/en-us/sql/t-sql/statements/insert-transact-sql?view=sql-server-ver16#using-insert-into-select-to-bulk-import-data-with-minimal-logging-and-parallelism>

- **ALTER TABLE** clause - <https://docs.microsoft.com/en-us/sql/t-sql/statements/alter-table-transact-sql>

Another powerful tool for importing large amounts of data into your tables is **BULK INSERT** T-SQL clause. You can use the **BULK INSERT** command if you want to import the entire CSV file into an existing table. The **BULK INSERT** T-SQL clause is used to efficiently import a large number of rows into a SQL Server table from a data file. It's a high-performance alternative to using individual **INSERT** statements, especially when dealing with large datasets stored in files like CSV or other delimited text files. This method requires that you have already created a table with a matching structure in your database.

The basic syntax of the **BULK INSERT** statement looks like this:

```
BULK INSERT TableName
FROM 'X:\\path\\file'
WITH ( -- parameters go here );
```

BULK INSERT key parameters:

- **FIELDTERMINATOR**: Specifies the character that separates fields (columns) in your data file (e.g., ',', '|', '\t').
- **ROWTERMINATOR**: Specifies the character that separates rows (records) in your data file (e.g., '\n', '\r\n').
- **FIRSTROW**: Specifies the row number in the data file to start importing from (useful for skipping header rows).
- **LASTROW**: Specifies the row number in the data file to stop importing at.
- **TABLOCK**: Specifies that a table-level lock is acquired for the duration of the bulk insert operation. This can improve performance but might block other users from accessing the table – but is it an issue in an OLAP system?
- **FORMAT**: Specifies the format of the data file (e.g., 'CSV').
- **BATCHSIZE**: Specifies the number of rows in a batch. Each batch is copied to the server as one transaction. If this fails, SQL Server commits or rolls back the transaction for every batch. By default, all data in the specified data file is one batch.
- **ERRORFILE**: Specifies the file used to collect rows that have formatting errors and can't be converted to an OLE DB rowset. These rows are copied into this error file from the data file "as is." The error file is created when the command is executed. An error occurs if the file already exists.
- **MAXERRORS**: Specifies the maximum number of syntax errors allowed in the data before the bulk-import operation is canceled.

Example of **BULK INSERT** statement reading data from a CSV file:

```
BULK INSERT Sales.Orders
FROM 'C:\Data\orders_import.csv'
WITH (
    FORMAT = 'CSV', FIRSTROW = 2, FIELDTERMINATOR = ',', ROWTERMINATOR = '\n', TABLOCK
);
```

Please note that in the example we skip the header row, use “,” as field delimiter, use “\n” as row delimiter and enforce **TABLOCK** to improve performance by using table lock.

Please note that, the user executing the **BULK INSERT** statement needs the **ADMINISTER BULK OPERATIONS** permission and **INSERT** permission on the target table. SQL Server needs to be able to access the data file. This often means the file needs to be on the server's local file system or a network share that the SQL Server service account has access to. You'll need to consider how to handle potential errors during the bulk insert process. Importantly, please ensure that the data types in your file are compatible with the data types of the columns in your target table. For more details look at: <https://learn.microsoft.com/en-us/sql/t-sql/statements/bulk-insert-transact-sql?view=sql-server-ver16>

For more advanced scenarios, such as filtering data during import or creating a new table on-the-fly, you can use the `OPENROWSET(BULK...)` command. You don't need to create a database table previously: the command will create it automatically once you run the query. However, you need to define a format file that will hold a table definition of a table to be created automatically:

```
SELECT <attribute_list>
INTO NewTableName
FROM OPENROWSET(BULK 'X:\path\file.csv',
    FORMATFILE = 'X:\path\format_file.fmt',
    FIRSTROW = 2
) AS source_alias;
```

This approach requires a format file that defines the structure of the CSV data. Two types of format files are supported for bulk exporting and importing: non-XML format files and XML format files.

A non-XML format file is a text file that has a specific structure. The non-XML format file contains information about the file storage type, prefix length, field length, and field terminator of every table column. The following illustration illustrates the format-file fields for a sample non-XML format file.

	Host file data type	Host file data length	Server column order	Column collation
Version	9.0			
Number of columns	4			
Host file field order	1	SQLCHAR 0 7	"\t" 1	DepartmentID ""
	2	SQLCHAR 0 100	"\t" 2	Name SQL_Latin1_General_CP1_CI_AS
	3	SQLCHAR 0 100	"\t" 3	GroupName SQL_Latin1_General_CP1_CI_AS
	4	SQLCHAR 0 24	"\r\n" 4	ModifiedDate ""

- *Host file data type* – Indicates the data type that is stored in a given field of the data file. With ASCII data files, use SQLCHAR.
- *Prefix length* – Number of length prefix characters for the field. Valid prefix lengths are 0, 1, 2, 4, and 8. To avoid specifying the length prefix, set this to 0.
- *Host file data length* – Maximum length, in bytes, of the data type stored in the particular field of the data file. For a delimited text file, you can specify 0 for the host file data length of every data field.
- *Server column order* – Order in which columns appear in the SQL Server table (1-wise). To omit a particular column, use 0.
- *Server column name* – Name of the column copied from the SQL Server table.

For details regarding the XML format please look at official documentation - <https://learn.microsoft.com/en-us/sql/relational-databases/import-export/xml-format-files-sql-server?view=sql-server-ver16>

Note: If you get the following error "SQL Server blocked access to STATEMENT 'OpenRowset/OpenDataSource' of component 'Ad Hoc Distributed Queries' because this component is turned off as part of the security configuration for this server. A system administrator can enable the use of 'Ad Hoc Distributed Queries' by using sp_configure. For more information about enabling 'Ad Hoc Distributed Queries', search for 'Ad Hoc Distributed Queries' in SQL Server Books Online.", please enable Ad Hoc Distributed Queries (which are disabled by default in SQL Server for security reasons). To resolve this issue use the following statements (first to enable advanced options – you might want to disable it later; second to enable ad hoc distributed queries):

```
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE WITH OVERRIDE;
GO
```

```
EXEC sp_configure 'ad hoc distributed queries', 1;
RECONFIGURE WITH OVERRIDE;
GO
```


For more information regarding OPENROWSET and bulk import of data look at: <https://learn.microsoft.com/en-us/sql/t-sql/statements/insert-transact-sql?view=sql-server-ver16#using-openrowset-and-bulk-to-bulk-import-data>

MERGE INTO is a Transact-SQL (T-SQL) statement is a tool for combining data from multiple sources into a single target table. It allows you to perform conditional insert, update, or delete operations on a target table based on the results of a join with a source table. It essentially combines these three data manipulation operations into a single, efficient statement.

Imagine you have two tables: a target table containing the current state of your data and a source table with updates, new records, or records to be deleted. Instead of writing separate INSERT, UPDATE, and DELETE statements to synchronize these tables, you can use the MERGE INTO statement. As such, common Use Cases include updating existing records and inserting new records from a staging table, synchronizing data between two tables, and performing so-called "upsert" operations (update if exists, insert if not).

The basic syntax of the MERGE INTO statement looks like this:

```
MERGE target_table_name AS Target
USING source_table_name AS Source
ON (join_predicate_expression)
WHEN MATCHED [AND condition] THEN
    -- Actions to perform when a matching row is found in both tables
WHEN NOT MATCHED BY TARGET [AND condition] THEN
    -- Actions to perform when a row exists in the source but not in the target
WHEN NOT MATCHED BY SOURCE [AND condition] THEN
    -- Actions to perform when a row exists in the target but not in the source
[OUTPUT $action, inserted.*, deleted.*];
```

Let us now do a quick statement breakdown:

- **MERGE INTO target_table_name AS Target**
 - Specifies the target table that will be modified
- **USING source_table_name AS Source**
 - Specifies the source table containing the data to compare against
- **ON (join_predicate_expression)**
 - Defines the matching condition between source and target tables (like a JOIN condition); typically compares key columns (e.g., 'ON Target.ID = Source.ID')
- **WHEN MATCHED THEN**
 - Defines actions when a matching row exists in both tables
 - UPDATE SET Target.column1 = Source.column1, Target.column2 = Source.column2
- **WHEN NOT MATCHED BY TARGET THEN**
 - Defines actions when a row exists in the source but not in the target (INSERT)
 - INSERT (column1, column2) VALUES (Source.column1, Source.column2)
- **WHEN NOT MATCHED BY SOURCE THEN**
 - Defines actions when a row exists in the target but not in the source (DELETE); often used to remove obsolete records from the target
 - DELETE
- **OUTPUT \$action, inserted.*, deleted**
 - Optional clause that returns information about the changes made, where '\$action' shows the operation performed (INSERT, UPDATE, DELETE), 'inserted.*' returns all columns from inserted/updated rows, and 'deleted.*' returns all columns from deleted/updated rows

Example

```
-- Create a temporary table variable to hold the output actions.
DECLARE @SummaryOfChanges TABLE (Change VARCHAR(20));

MERGE INTO Sales.SalesReason AS tgt
USING (
    VALUES ('Recommendation', 'Other'), ('Review', 'Marketing'), ('Internet', 'Promotion')
) AS src(NewName, NewReasonType)
ON tgt.Name = src.NewName
WHEN MATCHED THEN
    UPDATE
        SET ReasonType = src.NewReasonType
WHEN NOT MATCHED BY TARGET THEN
    INSERT (Name, ReasonType)
        VALUES (NewName, NewReasonType)
OUTPUT $action INTO @SummaryOfChanges;

-- Query the log results
SELECT Change, COUNT(*) AS CountPerChange
FROM @SummaryOfChanges
GROUP BY Change;
```

A thing to remember is that the `MERGE` statement is atomic, i.e., either all operations succeed or none do. Moreover, you can include multiple `WHEN MATCHED/WHEN NOT MATCHED` clauses with additional conditions in your query – as such have a more nuanced way of handling specific situations. In case of multiple `WHEN` clauses SQL Server executes them in order specified. Remember, the order of `WHEN` clauses matter. Additionally, you can omit any of the `WHEN` clauses if you don't need that operation and handling such a situation.

This statement is commonly used for synchronizing tables, slowly changing dimensions in data warehouses, and implementing upsert (insert-or-update) operations.

For more details look at `MERGE INTO` clause – <https://learn.microsoft.com/en-us/sql/t-sql/statements/merge-transact-sql?view=sql-server-ver16>

TASK 1 – SIMPLE QUERYING

We would like to study the performance of a query. As such, you will dive a bit into the area of query performance analysis. You will introduce three different SQL implementations, all capable of providing identical results, and study their performance. Prepare a short report: use prepared SQL queries, established execution plans, and your preliminary performance assessment (based on actual execution plans).

Compare performance of two queries, which support assessing the performance of individual sales representatives working in AdventureWorks company. The key metrics that we are focused on is the total sales made by employees. We want to identify salespeople that perform better at the end of the year. The report should represent pivoted data as shown in Table 1.

Table 1. Result structure for task 1-3

Sales Person	Quarter1	Quarter2	Quarter3	Quarter4
...	...	...	...	...

1. Prepare the report without using `PIVOT` (using only `CASE` clauses)
2. Prepare the report using `PIVOT` (no `CASE` clauses)
3. Prepare the report by using CTEs, where first you aggregate the sales data to establish quarterly performance metrics (one CTE per quarter), and only then join all the quarterly performances by sales representatives (do not use `OVER` nor `CASE` clause here).

Prepare a short report (using proper SQL queries) to assess the yearly performance of individual sales representatives working in AdventureWorks company. The key metrics that we are focused on are the total sales and number of orders made by employees. The report should contain data as shown in Table 2.

Table 2. Result structure for task 4-6

Sales Person	Employee ID	Year	Sub Total	Number of orders
...	...	...	...	...

4. Prepare the report without using windowed functions (no OVER clause)
5. Prepare the report using windowed functions (OVER clause)
6. Prepare the report by using CTE, where first you aggregate the sales data to establish yearly performance metrics, and only then attach the details of the salesperson (do not use OVER clause here).

TASK 2 – SCHEMAS, INGESTING CSV DATA AND MOVING DATA AROUND

Assume you need to prepare data for data analysts in Adventure Works Cycles Company, who are interested in analysing basic sales information from the product perspective. They will use basic reporting and data visualisation tools, like pivot tables in Excel and reports in PowerBI. As such your task is to prepare new dedicated relational data structures for basic OLAP processing and move the data from available sources using SQL operations.

In general, we want to follow a simple process, where the data (available in external sources) is first extracted (with no business logic applied in this step), then it is cleaned and conformed in the staging area (a dedicated data storage and processing solution), and is finally loaded into the dedicated data warehouse (a dedicated analytical data storage and processing solution). In our case, we will use a single SQL Server instance and emulate the warehouse layer using a schema. We will additionally, differentiate (withing the staging area) source/raw data and staging tables (used for intermediate steps in data transformation).

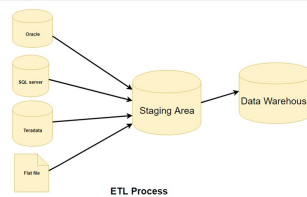


Figure 1. Overview of the storage layers in the data preparation process

Create schemas:

1. Prepare a new schema to handle the source data [SRC], schema to handle staging area data [STAGE] and a schema to handle final model [DW].

Extract source data:

2. Assume that your source data is available on the external server (details are available on ePortal) – do not use your local instance of AdventureWorks, use the external server. Extract this external data into your own database in a new dedicated schema [SRC] – destined to keep source data. Note, that you do not ingest the entire database, only needed data – as we are currently focusing on product information only limit your processing to product data (add SalesOrderDetail table).
 - a. Identify needed tables in AW database and extract them into your local database and [SRC] schema.
3. Plan how you could verify completeness and correctness of such an extraction process.

Next, still within the staging area, run proper transformation steps to convert data according to the business requirements. Reformat converted data to a standardized format for compatibility, cleanse irrelevant data from the datasets. Additionally, analyse the mirrored data for possible anomalies Try to identify basic problems with the data, i.e. that might cause troubles during data consumption. For instance, uniqueness of key values, wrong formatting, wrong encoding, missing values, etc. Finally, please verify data correctness and completeness, i.e., make sure you haven't left any of the data out. Please remember that the whole transformation is a process and doesn't have to be done using just a single SQL statement – it can be a series of statements, that each works on top of the other (or even in parallel), and intermediate results can be stored in dedicated storage solution utilised in the staging area.

Finally, load the data into the target tables – here your local database and [DW] schema.

Note that the final state of data should represent data ready for data consumption. Please remember, that each attribute in the dimension table can be used by the user to define a possible filter/grouping of facts – as such, each attribute is highly valuable. Moreover, please remember that these attributes are used as they are prepared – data transformation is limited to the staging area only.

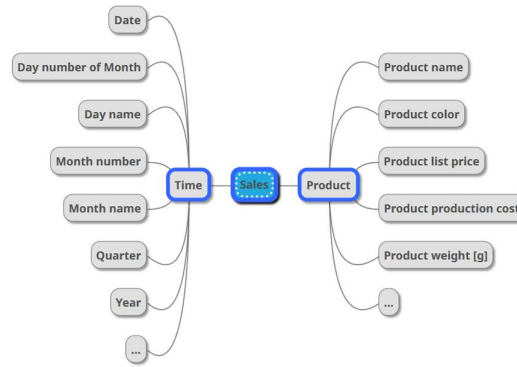


Figure 1. Overview of the resultant data model

Create a product perspective information:

4. Prepare needed queries (can be several) to move and merge all data about product (make sure to include all basic product information – include categories and subcategories) from AdventureWorks database. For this task use only main tables related to product information – note we are focusing on sales, so manufacturing details are not needed.
 - a. Use SELECT INTO to move data from SRC to STAGE in your own database.
 - b. Clean and standardize the data, if needed; use verbose entries (do not use flags, abbreviations, etc.); handle NULL values; do this within the [STAGE] schema.
 - c. DIMProduct– Product Dimension – with the following information:
 - i. Product ID,
 - ii. Basic information – name, product number, subcategory, category, list price
 - iii. Additional information – color, product line, class
 - iv. Specific information – weight, size
 - v. Calculated information - IsPurchasedAtleastOnce, IsManufactured
 - vi. Surrogate key that uniquely identifies a row within DimProduct
 - vii. Add other columns that might be needed.
5. Comment how you could verify completeness and correctness of such a transformation process.

Create a time/date perspective information:

6. Using Online sources (<https://www.mssqltips.com/sqlservertip/4054/creating-a-date-dimension-or-calendar-table-in-sql-server/>) create a 'dimDate' table in your database.
 - a. Start date – 01.01.2010; End date – 31.12.2020
 - b. Create additional column 'Date [EUStyle]' that would contain a Polish/European date format (DD-MM-YYYY)
 - c. Create additional column 'Day type' that would specify if a particular day is in weekend or a weekday.
 - d. Rename column 'Style112' as 'DateID' - set it as a composite Key – this will be used for joining other tables to dimDate.
 - e. Add column with day number of month and day of the week name, similar for month.
 - f. Add other columns that might be needed – look at your requirements from the previous task.
7. After creating the 'dimDate' table, verify the correctness of the table:
 - g. Check if the number of rows is correct (should be the number of days between the two dates)

- h. Check how many unique years are there in the table
- i. Check how many unique year-month combinations are there in the database
- j. Check 5 random rows for logical business correctness (data in all columns in a selected row should make logical sense)

Create a sales snapshot table:

8. Preparing data for analysis (OLAP) using SQL – Sales Information
 - a. SalesOrder ID, SalesOrderDetail ID (we will need them for future extensions to the model)
 - b. OrderDate, DueDate, ShipDate – convert it to a format that will match the dimDate.DateID attribute format
 - c. Add other columns that might be needed – look at your requirements from the previous task.
 - d. Product's surrogate key – make sure to match the correct product,
 - e. Basic measures – UnitPrice, Quantity, TotalValue
 - f. Additional measures – UnitPriceDiscount, TotalDiscountValue
 - g. Restore necessary constraints:
 - i. Primary Keys
 - ii. Foreign Keys – to DIMProduct
 - iii. Foreign Keys – to DIMTime
 - iv. Additional checks - if necessary

REMEMBER: To plan how you could verify completeness and correctness of such a process (*HINT – run basic checks, like counting the number of rows and/or calculating some aggregates*). Maybe you can automate this verification process for future data movements/loads.

REMEMBER: While preparing the data, remember that you should clean and standardize it – make sure it is “consumer ready”.

All results – SQL queries – should be stored in a single text file (sql). After completing the tasks, double check your results and please fill-in the report (use code and comments).

TASK 3 – ENRICH THE PRODUCT INFORMATION – DESIGN:

Using the already established model in the previous task we want to take advantage of the newly obtained data (about rating).

Think and design how can you incorporate the available information into the model (See Figure 1.). Note that the main goal of the analyst is to study sales (not the reviews themselves). Please update the model and justify your ideas/findings.

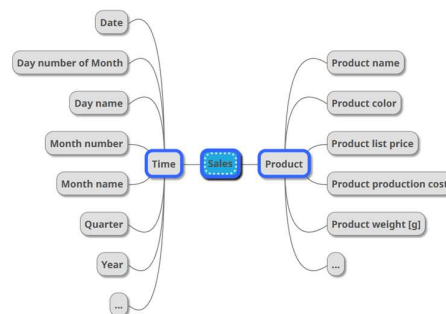


Figure 1. Overview of the resultant data model

1. Prepare an updated model – high-level design similar to Figure 1 (no implementation is needed here).

TASK 4 – ENRICH THE PRODUCT INFORMATION - IMPLEMENTATION:

Let us extract the collected information about products' ratings (both rating and location) from external CSV files (use DWI-LAB-Rating.csv, skip the NEW file for the time being):

1. Prepare a new schema to handle the external data and prepare needed queries to ingest the available data into your own database (you need write access).
 - a. Use appropriate approach to ingest data.
2. Move the data into staging area and process it using SQL – consider handling all quality issues (look at the previous task).
3. Comment how you could verify completeness and correctness of such a process

Now try to integrate products' ratings data with prepared product perspective.

4. We want to create a table containing all the product details
 - a. Use INSERT INTO to move data – can you follow the same process, as in case of SELECT INTO? Do you need to manually create the table structure? Are primary keys preserved? If not introduce primary keys.
5. Comment how you could verify completeness and correctness of such a process

Finally, we want to create a table representing the product information:

6. Finally load the data into the final structure - in short, we want to prepare a [DW].DIMProduct:
 - a. Move data – use your preferred method
 - b. Set necessary constraints – Primary Keys; Additional checks - if necessary
7. Comment how you could verify completeness and correctness of such a load process.

TASK 5 – ENRICH THE PRODUCT INFORMATION – INCREMENTAL:

Now try to incorporate the new, updated, information about products' ratings (both rating and location) from an external CSV file – DWI-LAB-RatingNEW.csv. This is a file that was obtained later and includes new ratings and updated data – note that some measures have changed.

1. Plan your approach to incorporate the new data into already existing data model.
2. Prepare needed queries to ingest the newly available data into your own database and integrate it with already existing data.
3. Comment how you could verify completeness and correctness of such a process

TASK 5 – ALTERNATIVE – INCREMENTAL EXTRACTION:

Proceed by inspecting the Production.Product_1, Production.Product_2, and Production.Product_3 tables within the external AdventureWorks database, which is provided on ePortal. Subsequently, develop an incremental strategy to incorporate the product data found in these tables into the target data model.

All results – SQL queries – should be stored in a single text file (sql). After completing the tasks, double check your results and please prepare a short report (use code and comments) with final conclusions.

REMINDER – QUIZ ASSIGNMENT

Quick task, done at the beginning of next week's class - details during class, scored as a separate assignment. Focused on SQL queries (pivots, windows, groupings, cases) using AdventureWorks database.